

DOCUMENTATION

Maw1.1 projet

Amin De Abreu Jdidi

Bryan Zweiacker

Aurélien Robert

Table des matières

Introduction.....	3
Pages principales.....	3
MCD / MLD.....	3
Structure du code et gestion de projet.....	4
Améliorations par rapport au site de base.....	4
Front-end.....	4
Back-end.....	4
Gestion des erreurs.....	5
Conventions de code, fichiers / dossiers et chemin.....	5
GitHub / Gitflow.....	5
IceScrum.....	6
CSS.....	6
CDN.....	6

Introduction

Looper est une plateforme web où les enseignants peuvent créer des exercices interactifs, et les étudiants y répondent. Les exercices ont un statut "building", "answering", ou "closed" pour contrôler l'accès, avec des vérifications sur les réponses pour éviter les bugs ou les réponses vides.

Pages principales

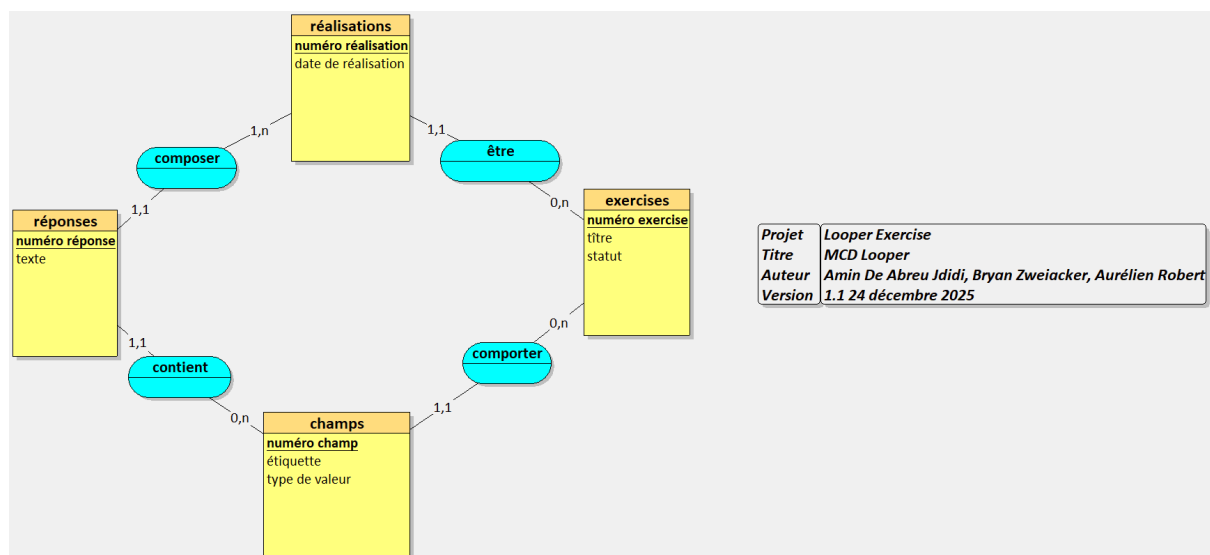
- Accueil : Liste des exercices disponibles
- Take an exercise : Sélectionner un exercice
- Create an exercise : Création d'un exercice
- Manage an exercise : Gestion/modification d'un exercice
- Complete an exercise : Réponse à l'exercice
- See all answers : Consultation des réponses

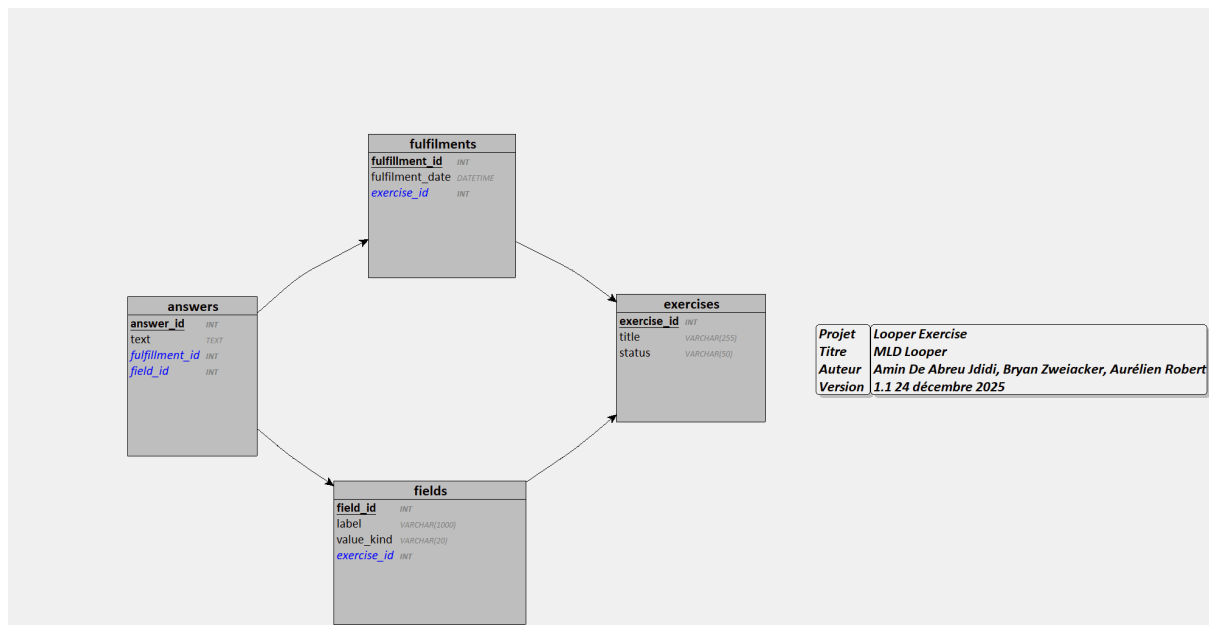
MCD / MLD

Le MCD et le MLD ci-dessous décrivent les entités principales du projet et leurs relations :

- **exercices** : exercice (titre, statut).
- **fields** : champs d'un exercice (label, type de valeur).
- **answers** : réponses données à un champ.
- **fulfilments** : réalisation complète d'un exercice à une date donnée.

Un exercice possède plusieurs champs, un fulfilment regroupe plusieurs réponses, et chaque réponse correspond à un champ précis d'un exercice.





Structure du code et gestion de projet

Nous avons opté pour une architecture **MVC web** pour la structure des dossiers et une méthode **agile** avec **IceScrum** pour la gestion de projet.

Améliorations par rapport au site de base

- Ajout de vérifications et amélioration de la gestion des données des formulaires.
- Pour la page **sauvegarder les réponses**, au moins **un champ doit être rempli** pour que les réponses soient prises en compte, côté **front-end** et **back-end**.

Front-end

- Limites de caractères.
- Placeholder pour aider l'utilisateur.
- Champ obligatoire si nécessaire.

Afficher les erreurs si on contourne les sécurités du front en rouge en dessous des champs

Back-end

- Vérification du nombre de caractères et des champs vides.

Sur les pages suivantes :

Page de création d'un exercice (titre).

Page de modification d'un exercice :

- Possibilité d'ajouter des champs.
- Modification d'un champ déjà créé.

Page de sauvegarde des réponses des étudiants.

Dans la page d'édition d'un champ, on utilise un alias (single line text) au lieu d'afficher directement la valeur qui est dans la base de données(single_line).

Gestion des erreurs

- Ajout d'une **erreur 404** pour toutes les pages censées être inaccessibles (non présentes dans le dispatcher), par exemple :
`localhost:8888/errorhgzg`

Gestion spécifique pour les pages dont l'accès est interdit selon leur **statut** via l'URL, par exemple :

- essayer d'accéder à l'édition ou à la réponse d'un exercice "**closed**"
- essayer d'accéder aux réponses d'un exercice qui n'est pas en "**answering**"

Conventions de code, fichiers / dossiers et chemin

Nommage respectant PSR-12 :

- **Fichiers et dossiers** : `Model/DataBase.php`
- **Fonctions** : camelCase, par exemple : `function funTest()`.
- **Variables** : par exemple : `$var1`.

Exemple :

```
function saveExercise($exerciseData) {  
    $validatedData = validate($exerciseData);  
}
```

GitHub / Gitflow

Pour le code, chaque feature a été séparée par branche à l'aide de Gitflow.

```
git flow feature start nom_feature
```

IceScrum

Concernant la gestion de projet, nous avons utilisé IceScrum pour répartir les différentes tâches et faciliter le suivi du projet. Nous avons travaillé par sprint de deux ou trois semaines. Les différentes tâches ont été réparties par difficulté et valeur.

CSS

Un CSS commun gère la base pour toutes les pages du site. Un CSS spécifique peut-être créé, comme pour home, qui possède des éléments qui lui sont propres.

CDN

Nous utilisons Font Awesome pour afficher les icônes de l'application. Plutôt que de télécharger la bibliothèque complètement en local, nous l'incluons via le CDN CDJS hébergé par CloudFlare.

Justification : Notre application n'utilise qu'un nombre limité d'icônes. Un CDN présente plusieurs avantages dans ce contexte :

- Pas de fichiers supplémentaires à maintenir dans le projet.
- Mise en cache potentielle si l'utilisateur a déjà visité d'autres sites utilisant le même CDN.
- Réduction de la taille du dépôt Git.